Отчёт по практике 4

Разработка безопасного программного обеспечения: аутентификация

Выполнил: Колесников Андрей Юрьевич, 4 курс, группа БСБО-03-23, студенческий билет 23Б0099, номер по журналу 10

РТУ МИРЭА, Институт кибербезопасности и цифровых технологий

1. Цель работы

Работа состоит из трёх связанных частей. Сначала нужно было написать переборщик для формы входа из учебного стенда DVWA и посмотреть, сколько времени занимает подбор пароля по словарю. Затем — разобрать исходный код этой формы и классифицировать найденные слабости по базе CWE. И наконец — сделать свою систему аутентификации, которая сохраняет формат запроса из задания, но устойчива к перебору и к остальным проблемам исходного кода.

2. Стенд и условия выполнения

В методичке предлагается Web Security Dojo через VirtualBox. Я использовал обычный DVWA в Docker: это тот же самый модуль vulnerabilities/brute с тем же кодом, но без необходимости разворачивать виртуальную машину на 4 ГБ. Версия DVWA — 1.10, она поднята на 127.0.0.1:8081. Пароль администратора стенда изменён на Summer2024!, чтобы он не совпадал с паролем по умолчанию и его можно было искать по словарю (в словаре он стоит двадцатым). Это изменение касается только локального учебного стенда.

Собственное приложение написано на PHP 8.3 и работает на встроенном сервере PHP в контейнере php:8.3-cli, база — SQLite через PDO. Такой выбор позволил не ставить PHP и MySQL на хост и получить одинаковое окружение при каждом запуске. Переборщик запускается на хосте под Python 3.12.4 и обращается к стенду по HTTP. Оба адреса — локальные, внешние системы не затрагивались.

Компонент	Значение
Хост	Windows 11, AMD Ryzen 7 5700U (8 ядер / 16 потоков), 16 ГБ ОЗУ
DVWA	Docker-образ vulnerables/web-dvwa, http://127.0.0.1:8081, уровень high
Своё приложение	PHP 8.3, PDO/SQLite, http://127.0.0.1:8080
Переборщик	Python 3.12.4, библиотека requests
Словарь	wordlist_p4.txt, 30 паролей, пароль стенда — 20-й

3. Переборщик для DVWA (пункт 1)

Скрипт bruteforce_dvwa.py работает так. Сначала он либо входит в DVWA по логину и паролю администратора, либо принимает готовую cookie через параметр --cookie. Уровень защиты задаётся cookie security=high. Затем для каждого пароля из словаря скрипт отдельным GET-запросом открывает страницу формы и забирает из неё актуальный user_token, после чего отправляет проверочный запрос в формате из задания:

GET /vulnerabilities/brute/?username=USER&password=PASS&user_token=TOKEN&Login=Login

Ответ разбирается по тексту: фраза «Welcome to the password protected area» означает успех, «Username and/or password incorrect» — неудачу, сообщение о блокировке или код 429 — что стенд прекратил принимать попытки, отдельное сообщение о неверном токене — что нужно перезапросить страницу. Если токен не подошёл, скрипт один раз повторяет попытку с новым токеном. Результат пишется в CSV: адрес, логин, число попыток, найденный пароль, время и статус.

Для защиты от случайного запуска по чужому адресу в скрипте есть проверка цели: разрешены localhost, петлевые и частные адреса, а также имена вида *.local. Всё остальное отклоняется с кодом возврата 2, пока не указан флаг --allow-remote. Это именно предохранитель, а не полноценная защита — при желании флаг можно обойти, но случайно вставить в команду адрес реального сайта уже не получится.

Запуск против DVWA:

python bruteforce_dvwa.py --url http://127.0.0.1:8081/vulnerabilities/brute/ --username admin --wordlist wordlist_p4.txt --login-url http://127.0.0.1:8081/login.php --login-username admin --login-password Summer2024! --security high --results dvwa_results.csv

Результат: пароль найден на 20-й попытке за 28,42 с. В среднем одна попытка заняла 1,42 с. Это время почти целиком состоит из задержки самого стенда: на уровне high DVWA при неудачном входе выполняет sleep(rand(0,3)), то есть в среднем добавляет около полутора секунд. Сам MD5 и сетевой запрос на этом фоне теряются: миллисекунды против секунд.

Цель	Уровень	Попыток	Результат	Время, с
DVWA, модуль bruteforce	high	20	найден пароль Summer2024!	28,42
SecureAuthApp, форма входа	—	6	сработала блокировка, пароль не найден	6,22

Отдельно стоит сказать про user_token. На первый взгляд он выглядит как защита от перебора, но от перебора он не защищает совсем: это CSRF-токен, он нужен, чтобы чужой сайт не мог отправить форму от имени пользователя. Скрипт — не чужой сайт, а обычный HTTP-клиент: он честно заходит на страницу и получает свежий токен перед каждой попыткой, как это сделал бы браузер. Поэтому наличие токена меняет только количество запросов (два вместо одного), но не останавливает подбор.

4. Код-ревью формы входа (пункт 2)

Код модуля bruteforce в DVWA отличается по уровням. На low нет вообще ничего: ни экранирования, ни токена, ни задержки. На medium добавлен mysqli_real_escape_string для логина и пароля и sleep(2) при неудаче. На high к этому добавлены stripslashes, проверка user_token через checkToken и sleep(rand(0,3)). При этом SQL-запрос на всех трёх уровнях по-прежнему собирается конкатенацией строк, а пароль по-прежнему хешируется через md5(). Именно это и показывает, что «высокий» уровень в учебном приложении означает лишь усложнение эксплуатации, а не исправление архитектуры.

CWE	Где проявляется	В чём проблема
CWE-89 SQL injection	SELECT * FROM users WHERE user = '$user' AND password = '$pass'	Логин и пароль подставляются в текст SQL. На low ввод вида ' OR '1'='1' -- обходит проверку пароля. На medium/high используется mysqli_real_escape_string, но это ручное экранирование под конкретный драйвер, а не подготовленные выражения; правильное решение — параметры запроса.
CWE-327 / CWE-916 Слабый алгоритм хеширования	$pass = md5($pass);	MD5 быстрый, без соли и без параметра стоимости. Одинаковые пароли дают одинаковые хэши, а скорость вычисления делает словарь и радужные таблицы очень эффективными.
CWE-307 Нет ограничения попыток	Весь обработчик формы	Ни счётчика неудач, ни блокировки, ни капчи. sleep(2) на medium и sleep(rand(0,3)) на high только замедляют перебор, но не ограничивают число попыток: скрипт просто работает дольше и всё равно доходит до конца словаря.
CWE-598 Пароль в GET-запросе	username, password, user_token,	Пароль попадает в историю

	Login в URL	браузера, журналы веб-сервера и прокси, а также может уйти в заголовке Referer. Для формы входа нужен POST с телом запроса.
CWE-209 Утечка через сообщение об ошибке	mysqli_query(...) or die(mysqli_error(...))	При ошибке SQL наружу выводится текст ошибки СУБД. Это раскрывает структуру таблиц и помогает строить дальнейшие инъекции.
CWE-79 XSS	$html .= "... {$user}"; $html .= "";	Логин и аватар вставляются в HTML без htmlspecialchars(). Для эксплуатации нужен пользователь с HTML в имени или аватаре, поэтому это скорее отложенный дефект, но правильное экранирование обязательно.
CWE-1004 / CWE-614 Cookie без защиты	Set-Cookie: PHPSESSID=...; path=/	У сессионной cookie нет HttpOnly, SameSite и Secure. Её можно прочитать из JavaScript при XSS и отправить с чужого сайта; по HTTPS она тоже пойдёт без признака Secure.
CWE-693 Нет защитных заголовков	HTTP-ответы DVWA	Нет X-Frame-Options, Content-Security-Policy и X-Content-Type-Options. Страницу можно встроить в iframe (кликджекинг), а браузеру не запрещено угадывать MIME-тип.

CSRF-токен на high закрывает CWE-352, и это единственное серьёзное исправление по сравнению с low. Но к перебору оно отношения не имеет: токен проверяет источник запроса, а не количество попыток. Поэтому high уровень и был выбран для перебора — он показывает, что даже «самый защищённый» вариант DVWA всё равно подбирается словарём.

5. Собственная система аутентификации (пункт 3)

Приложение лежит в папке SecureAuthApp и не использует ни фреймворков, ни Composer — только PHP и расширение PDO SQLite. Точки входа находятся в public/: index.php, register.php, login.php, dashboard.php и logout.php. Общая настройка сессии, заголовков и обработки ошибок вынесена в src/bootstrap.php, чтобы её нельзя было забыть на отдельной странице. Работа с БД — в src/db.php, CSRF — в src/csrf.php, учёт попыток

— в src/rate_limit.php, настройки — в src/config.php. Схема таблиц описана в schema.sql, а тестового пользователя удобно создавать через tools/seed_user.php.

Форма входа намеренно оставлена GET-запросом с теми же именами параметров, что и в задании: username, password, user_token, Login. Это требование пункта 3, и оно же является единственным сознательно оставленным компромиссом — про него ниже, в разделе остаточных рисков. Регистрация и выход, где совместимость с чужим форматом не требуется, сделаны через POST.

Проблема	Что сделано в SecureAuthApp
CWE-89, SQL-инъекция	Все запросы идут через PDO с подготовленными выражениями; PDO::ATTR_EMULATE_PREPARES выключен, поэтому параметры передаются драйверу отдельно от текста SQL. Ввод не склеивается с запросом ни в login.php, ни в rate_limit.php.
CWE-327 / CWE-916, слабый хэш	password_hash() и password_verify() с алгоритмом по умолчанию (bcrypt, cost 10). Соль генерируется автоматически и хранится внутри хэша; при входе выполняется password_needs_rehash() на случай смены параметров.
CWE-307, перебор	Не более 5 неудачных попыток за 15 минут отдельно по логину и по IP. При превышении — ответ 429 с Retry-After. Дополнительно каждая попытка задерживается на 1 секунду, чтобы перебор не был мгновенным даже до срабатывания блокировки.
Раскрытие существования логина	Для несуществующего пользователя всё равно вычисляется bcrypt от хэша-заглушки, а текст ошибки одинаковый. По времени ответа отличить существующий логин от несуществующего нельзя.
CWE-352, CSRF	user_token — 32 случайных байта в сессии, сравнение через hash_equals(). Токен обязателен для входа, регистрации и выхода. Выход сделан POST-запросом, чтобы ссылка с чужого сайта не разлогинивала пользователя.
CWE-384, session fixation	После успешной проверки пароля вызывается session_regenerate_id(true): идентификатор, который был до входа, перестаёт действовать.
CWE-1004 / CWE-614, cookie	Своё имя SECUREAUTHSESSID, HttpOnly, SameSite=Lax, Secure при HTTPS, session.use_strict_mode=1 и session.use_only_cookies=1. Идентификатор сессии не передаётся в URL.
CWE-79, XSS	Все данные, которые выводятся на страницу, проходят через htmlspecialchars(). Плюс Content-

	Security-Policy default-src 'self' — встроенных скриптов на страницах нет вовсе.
CWE-209, утечка ошибок	display_errors выключен, необработанные исключения перехватываются общим обработчиком: клиент видит только «Внутренняя ошибка», а подробности уходят в журнал PHP.
CWE-693, заголовки	X-Frame-Options: DENY, Content-Security-Policy с frame-ancestors 'none', X-Content-Type-Options: nosniff, Referrer-Policy: no-referrer, Cache-Control: no-store. Заголовок X-Powered-By удалён.
Обрезание bcrypt	Пароль длиннее 72 байт отклоняется и при регистрации, и при входе. Иначе два разных длинных пароля с одинаковым началом считались бы одним и тем же.

Блокировка устроена на таблице login_attempts: каждая попытка пишется с логином, IP, признаком успеха и временем. Для проверки берутся неудачные записи за последние 15 минут; если их не меньше пяти, вычисляется момент, когда самая старая из этих пяти выйдет за окно, и разница отдаётся в Retry-After. Проверка выполняется до обращения к таблице users, поэтому заблокированный запрос не тратит время на bcrypt. Индексы по (username, attempted_at) и (ip_address, attempted_at) не дают этой проверке становиться узким местом.

6. Проверка работы

Все проверки выполнялись на локальном стенде. Для входа использовался пользователь testuser с паролем Summer2024!, созданный через tools/seed_user.php; перед проверкой блокировки журнал попыток очищался флагом --reset. Команды и результаты ниже приведены по фактическим ответам сервера, а не по ожиданиям.

Проверка	Результат
Успешный вход	302 на dashboard.php; идентификатор сессии изменился; dashboard отдаёт 200 и имя пользователя
dashboard без входа	302 на login.php
Неверный пароль, попытки 1–5	200, одинаковый текст «Username and/or password incorrect»
Шестая попытка	429, Retry-After: 895
Вход без user_token	200, «Сессия устарела», до БД дело не доходит
Вход с чужим user_token	200, «Сессия устарела»
user_token в виде массива	200, отказ без ошибки 500
SQL-инъекция ' OR '1'='1' -- в логине	200, общее сообщение об ошибке; dashboard по-прежнему 302, входа нет
XSS-логин <script>alert(1)</script>	200, сообщение об ошибке; в HTML попал

	<script>, сырого тега нет
Пароль короче 8 символов	200, «Пароль должен быть не короче 8 символов»
Пароль длиной 73 байта	200, «Пароль не должен быть длиннее 72 байт»
Выход GET-запросом	405, заголовок Allow: POST
Выход POST без токена	403
Выход POST с токеном	302 на login.php; dashboard после этого снова 302
Заголовки ответа	X-Frame-Options: DENY, CSP, Cache-Control: no-store, X-Powered-By отсутствует
Cookie сессии	SECUREAUTHSESSID; HttpOnly; SameSite=Lax
Хэш в базе	bcrypt, префикс $2y$10$; открытого пароля в БД нет
Переборщик по своей форме	6 попыток, статус locked_out, пароль не найден, 6,22 с
Проверка локальности цели	https://example.com/login отклонён, код возврата 2

Отдельно показателен запуск того же переборщика по своей форме: он делает 6 попыток и останавливается на блокировке, хотя искомый пароль лежит в словаре двадцатым. Время 6,22 с — это пять попыток с секундной задержкой и шестая, уже отклонённая. То есть защита работает не за счёт того, что пароль стал «сложным»: пароль остался тем же, изменилось лишь то, сколько попыток сервер готов принять.

Чтобы проверить защиту от определения логина по времени, я сделал по три замера для существующего и несуществующего пользователя. Существующий: 1,186; 1,212; 1,204 с, среднее 1,201 с. Несуществующий: 1,214; 1,195; 1,178 с, среднее 1,196 с. Разница около 0,005 с, то есть меньше половины процента и в пределах шума. Это ожидаемо: в обоих случаях выполняется bcrypt, просто во втором — от хэша-заглушки.

7. Ограничения и остаточные риски

Главный сознательный компромисс — пароль по-прежнему передаётся в GET-запросе. Этого требует пункт 3 задания, но с точки зрения безопасности это CWE-598: пароль может остаться в истории браузера, в access.log веб-сервера и в журнале прокси. В реальном проекте форму нужно переводить на POST, и я бы так и сделал; здесь формат сохранён только ради совместимости с проверкой из методички.

Блокировка по IP в текущем виде рассчитана на прямое подключение. Если приложение поставить за обратный прокси или балансировщик, REMOTE_ADDR будет адресом прокси, и все пользователи попадут в один общий счётчик. В таком случае нужно явно настраивать доверенный прокси и брать адрес из X-Forwarded-For, причём только из доверенного источника. В учебном стенде соединение прямое, поэтому этой проблемы нет.

Жёсткая блокировка по логину создаёт и обратную проблему: зная чужой логин, можно намеренно сделать пять неудачных попыток и временно закрыть владельцу вход.

Это цена простоты. В нагруженной системе вместо жёсткой блокировки обычно применяют прогрессивную задержку, CAPTCHA и оценку риска, а блокировку оставляют как крайнюю меру. Для лабораторной работы окно в 15 минут и пять попыток — разумный компромисс между защитой и удобством проверки.

Ограничения стенда: SQLite и файловые сессии рассчитаны на один узел, HTTPS в лаборатории не настроен (поэтому флаг Secure в cookie не появляется), а распределённый перебор с множества IP-адресов лимит по одному IP полностью не останавливает. Эти ограничения не мешают продемонстрировать основной механизм, но в реальной системе их пришлось бы решать отдельно.

8. Выводы

Перебор DVWA на высоком уровне показал главное: CSRF-токен и случайная задержка не являются защитой от подбора пароля. Скрипт получает токен точно так же, как браузер, а sleep лишь растягивает время. Пароль Summer2024! был найден на 20-й попытке за 28,42 с — при словаре из 30 слов это фактически полный успех. Причина не в слабом пароле как таковом, а в том, что форма принимает неограниченное число попыток и хранит пароль в виде быстрого MD5.

Собственная система на тех же входных данных ведёт себя иначе: после пяти неудач она отвечает 429 с Retry-After, и переборщик останавливается на шестой попытке, не дойдя до правильного пароля. Это достигается сочетанием трёх мер: bcrypt с cost 10 удорожает каждую проверку, секундная задержка не даёт делать тысячи запросов в секунду, а блокировка по логину и IP ограничивает общее число попыток. По отдельности ни одна из них не решает задачу: bcrypt можно перебирать годами, задержку — распараллелить, а блокировку — обойти сменой IP. Работает именно комбинация.

Сопутствующие проблемы исходного кода тоже закрыты: подготовленные выражения вместо конкатенации, password_hash вместо md5, экранирование вывода, CSRF-токен, смена идентификатора сессии после входа, защитные заголовки и одинаковый по времени ответ для существующего и несуществующего логина. Проверки из раздела 6 подтверждают каждую из этих мер отдельно, а не только общий результат.

Что можно улучшить дальше: заменить bcrypt на Argon2id с параметрами памяти (в PHP 8 это PASSWORD_ARGON2ID), добавить второй фактор для ответственных действий, ввести CAPTCHA после нескольких неудач и вынести счётчики попыток в общее хранилище вроде Redis, если приложение будет работать на нескольких серверах. Но и текущий вариант уже показывает разницу между «формой, которую видно в браузере» и формой, которую действительно можно защитить.

9. Состав файлов

bruteforce_dvwa.py — переборщик DVWA с проверкой локальности цели;

wordlist_p4.txt — словарь из 30 паролей;

dvwa_results.csv, secure_results.csv — результаты обоих запусков;

SecureAuthApp/public/ — страницы index, register, login, dashboard, logout;

SecureAuthApp/src/ — config, bootstrap, db, csrf, rate_limit;

SecureAuthApp/schema.sql — схема users и login_attempts;

SecureAuthApp/tools/seed_user.php — создание тестового пользователя;

gen_report.py — сборка этого отчёта из CSV.